

Esercizio A1 – Complessità – Punti ≥ 8

È facile vedere che $T(n)$ è $\Omega(n^3)$, per via della parte non ricorsiva. Verifichiamo se $T(n)$ è anche $O(n^3)$.

- **Caso base:** per $n = 1, \dots, 5$, si deve dimostrare che $T(n) = 1 \leq cn^3$; questa disequazione è vera per qualunque $c \geq 1$, $c \geq 1/8$, $c \geq 1/27$, $c \geq 1/64$, $c \geq 1/125$ rispettivamente; è quindi vera per ogni $c \geq 1$.
- **Ipotesi induttiva:** ipotizziamo che per ogni valore k , $1 \leq k < n$: $T(k) \leq ck^3$;
- **Passo induttivo:** consideriamo i casi con $n \geq 6$, per i quali $\lfloor n/3 \rfloor$ e $\lfloor n/6 \rfloor$ sono maggiori o uguali a 1 (il primo caso base); applichiamo la sostituzione e otteniamo:

$$\begin{aligned}
 T(n) &= 9T(\lfloor n/3 \rfloor) + 36T(\lfloor n/6 \rfloor) + n^3 \\
 &\leq 9c\lfloor n/3 \rfloor^3 + 36c\lfloor n/6 \rfloor^3 + n^3 && \text{Sostituzione} \\
 &\leq 9cn^3/27 + 36cn^3/216 + n^3 && \text{Eliminazione } \lfloor \rfloor \\
 &\leq cn^3/3 + cn^3/6 + n^3 && \text{Passaggio algebrico} \\
 &= \frac{1}{2}cn^3 + n^3 \stackrel{?}{\leq} cn^3 && \text{Passaggio algebrico}
 \end{aligned}$$

L'ultima disequazione è vera per $c \geq 2$.

Abbiamo quindi dimostrato che $T(n) = O(n^3)$, per $m = 1$ e $c = 2$. Poiché $T(n)$ è anche $\Omega(n^3)$, abbiamo che $T(n) = \Theta(n^3)$.

Esercizio A2 – Perfect inside – Punti ≥ 10

La funzione principale richiama una funzione ricorsiva che prende in input un nodo t e restituisce due valori: l'altezza del più grande albero binario perfetto che ha radice nel nodo t e l'altezza del più grande albero binario perfetto contenuto nel sottoalbero radicato in t .

Per calcolare l'altezza del più grande albero perfetto radicato in t , prende il minimo fra le altezze dei più grandi alberi perfetti radicati nei suoi figli sinistro e destro, e somma 1. Nel caso un nodo sia **nil**, restituisce -1 come altezza.

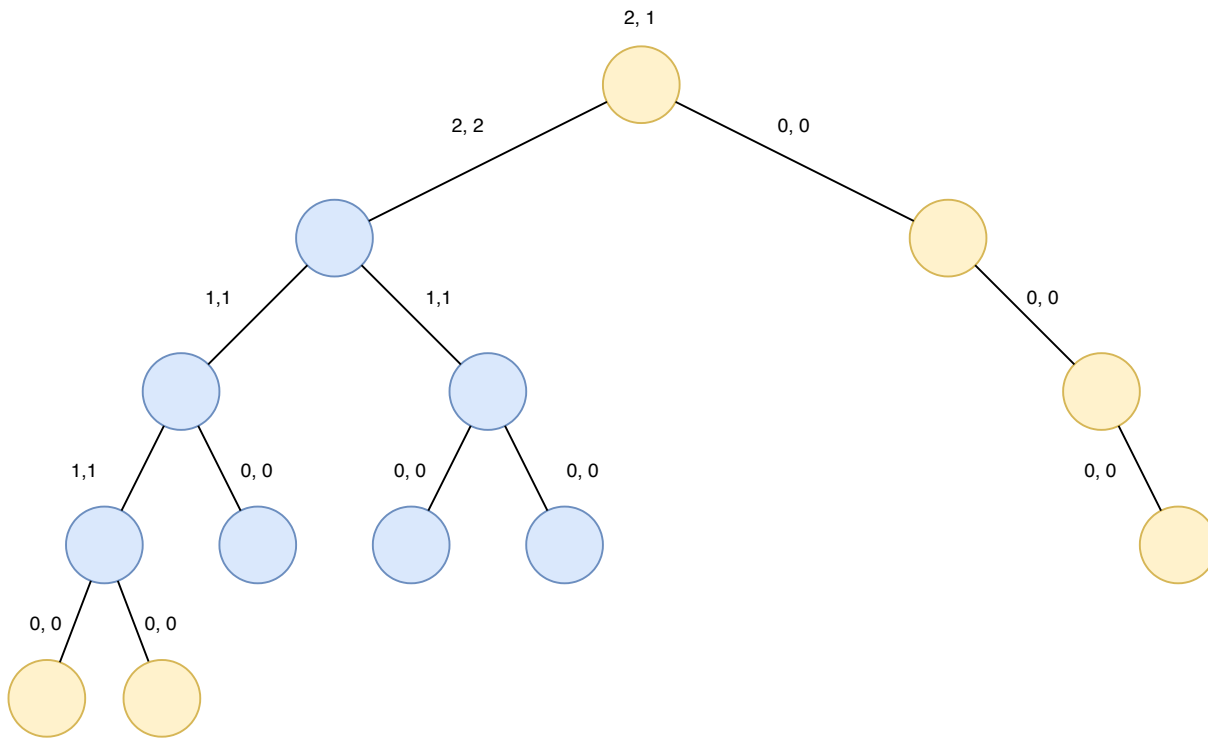
Vediamo alcuni casi particolari: se un nodo è una foglia, le altezze che ottiene dall'esecuzione ricorsiva sui due figli **nil** sono entrambe -1; quindi il minimo di tali valori + 1 è pari a zero, che è l'altezza di un nodo singolo. Se un nodo ha un solo figlio, l'altro è **nil**, quindi l'altezza è ancora 0.

Per calcolare l'altezza del più grandi albero perfetto contenuto in t , possiamo prendere semplicemente il massimo fra l'altezza dei più grandi alberi perfetti contenuti nei figli sinistro e destro e l'altezza del sottoalbero più grande radicato in t .

La complessità computazionale è ovviamente $O(n)$, in quando si tratta di una post-visita di un albero.

int largestPerfect(TREE T)
int $maxSoFar$, $current = \text{lpRec}(T)$ return $maxSoFar$
int , int lpRec(TREE t)
if $t == \text{nil}$ then return 0, -1 else int , int $maxSoFar_L$, $current_L = \text{lpRec}(t.\text{left})$ int , int $maxSoFar_R$, $current_R = \text{lpRec}(t.\text{right})$ int $current = \min(current_L, current_R) + 1$ return $\max(maxSoFar_L, maxSoFar_R, current)$, $current$

L'esecuzione dell'algoritmo sull'albero di esempio è mostrata nella figura sottostante, dove il valore di ritorno di un nodo è mostrato nell'arco che conduce al genitore (o sopra di esso nel caso della radice).



Esercizio A3 – Island Hopping – Punti ≥ 12

Il problema può essere risolto in modo semplice con quattro cicli annidati, che confrontano la distanza fra ogni cella con valore pari a 1 da ogni cella con valore pari a 2.

```
int distance(int[][] M, int n)
```

```
    int minSoFar
    for  $x_1 = 1$  to  $n$  do
        for  $y_1 = 1$  to  $n$  do
            if  $M[x_1][y_1] == 1$  then
                for  $x_2 = 1$  to  $n$  do
                    for  $y_2 = 1$  to  $n$  do
                        if  $M[x_2][y_2] == 2$  then
                             $minSoFar = \max(minSoFar, |x_1 - x_2| + |y_1 - y_2|)$ 
    return minSoFar
```

Questa soluzione ha costo $O(n^4)$. Per trovare una soluzione più efficiente, effettuiamo una visita in ampiezza a partire da tutte le celle con valore 1 (a cui assegniamo una distanza 0 e inseriamo tutte insieme nella coda). Identificare tutte le celle con valore 1 costa $O(n^2)$, effettuare la visita costa $O(n^2)$, quindi l'algoritmo ha costo totale $O(n^2)$.

```
int distance(int[][] M, int n)
```

```
    QUEUE Q = Queue()
    int[][] dist = new int[1...n][1...n] = {-1}
    for x1 = 1 to n do
        for y1 = 1 to n do
            if M[x1][y1] == 1 then
                Q.enqueue((x1, y1))
                dist[x1][y1] = 0
    int[] dx = [-1; 0; +1; 0]
    int[] dy = [0; +1; 0; -1]
    while not Q.isEmpty() do
        int, int x, y = Q.dequeue()
        for i = 1 to 4 do
            int nx = x + dx[i]
            int ny = y + dy[i]
            if 1 ≤ nx ≤ n and 1 ≤ ny ≤ n and dist[nx][ny] < 0 then
                dist[nx][ny] = dist[x][y] + 1
                if M[nx][ny] == 2 then
                    return dist[nx][ny]
                Q.enqueue((nx, ny))
    return +∞
```

Esercizio B1 – SWERC – Punti ≥ 8

Risolviamo il problema tramite una rete di flusso.

Aggiungiamo i seguenti nodi

- Una sorgente e un pozzo (2 nodi)
- Un nodo per ogni università (n_u nodi)
- Un nodo per ogni coppia (tavolo, paese) ($6n_t$ nodi)
- Un nodo per ogni tavolo (n_t nodi)

Aggiungiamo i seguenti archi:

- Un arco dalla sorgente s all'università U_i , con capacità n_i , per limitare il numero di studenti partecipanti dell'università da assegnare ai tavoli.
- Un arco dall'Università U_i ad ogni nodo $(T_j, P(U_i))$, con capacità 1. In pratica, al massimo uno studente può essere assegnato a un qualunque tavolo, selezionando però la coppia che associa il tavolo al paese dell'università.
- Un arco da ogni nodo (T_j, p) ad ogni tavolo T_j , con capacità 3, ad indicare che ogni paese p può contribuire con al massimo 3 studenti ad ogni tavolo T_j .
- Un arco da ogni nodo T_j al pozzo t , con capacità k , per limitare il numero di studenti per tavolo.

Nella figura seguente, trovate una rete con 4 università e 2 tavoli.